CFP/UAX
Centro de Formación Profesional

CLASIFICACIÓN EXOPLANETAS Y PLANETAS SEGÚN SU POTENCIAL HABITABILIDAD

PROYECTO FINAL ESPECIALIZACION BIG DATA & IA

Jorge Pizarro Gómez

Año: 2026

Índice

Comprensión del negocio 3
Entendimiento de los datos 3
Preparación de los datos 14
Modelado 17
Evaluación 21
Deployment 24
Bibliografía 26

Comprensión del negocio

- Dataset empleado: https://www.kaggle.com/datasets/niccosimonetti/nasa-exoplanet-archive

Este recopila parámetros planetarios físicos usados en astrobiología para estimar la habitabilidad potencial de planetas y exoplanetas.

El núcleo del proyecto se basará en el ESI (Índice de Similitud Terrestre), es un valor comprendido entre el 0 y el 1. La Tierra y sus características son la referencia para el valor máximo, el 1, según el valor sea más cercano a este significa que las características de ese cuerpo planetario son más similares a las condiciones terrestres por lo que sería más plausible que ese planeta pudiera o podría ser apto para la vida.

El ESI se puede calcular utilizando cuatro parámetros físicos de un planeta:

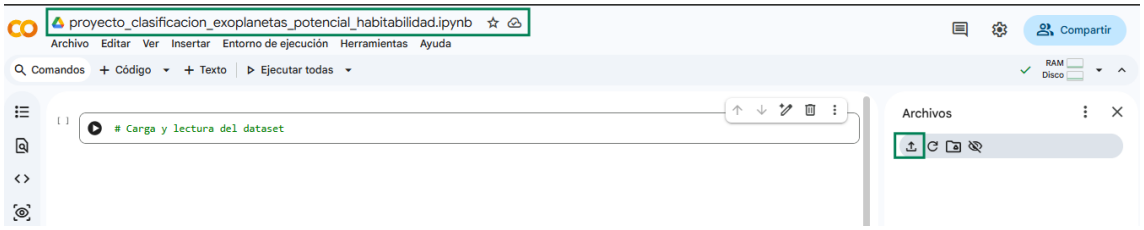
- Radio
- Densidad
- Velocidad de escape
- Temperatura superficial de equilibrio

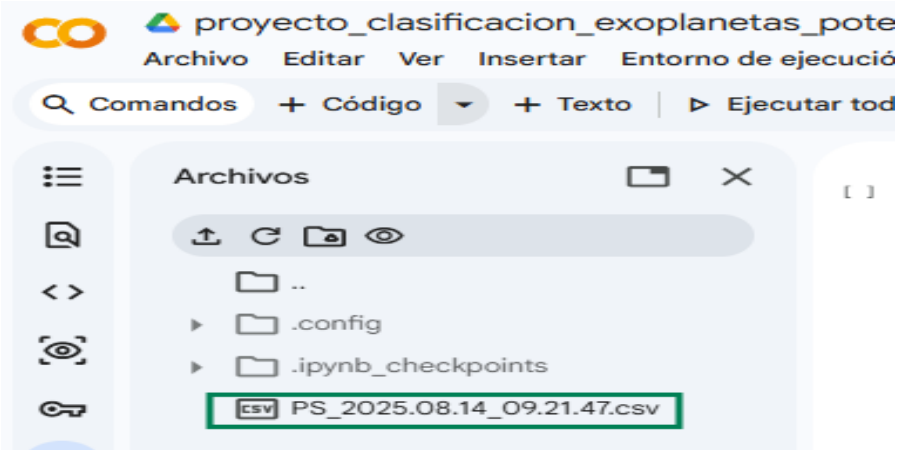
Otros conceptos de gran importancia sobre los cuales también girará entorno el proyecto es la habitabilidad y zonas habitables. Estos, hacen referencia a la región “alrededor de una estrella donde un planeta podría potencialmente mantener agua líquida en su superficie, un criterio clave para la habitabilidad tal como la entendemos”.

La investigación sobre la habitabilidad de exoplanetas examina la posición relativa de los planetas respecto a la zona habitable y otras propiedades físico-orbitales, usando datos del NASA Exoplanet Archive para caracterizar mundos con potencial para sostener condiciones favorables. Esto será justamente el objetivo de este caso de uso, entrenar un modelo de IA capaz de dilucidar entre los datos provenientes del dataset que planetas tienen mayor grado de similitud con a las características de la Tierra y por tanto es posible que pudieran albergar algún tipo de vida tal y como es conocida en nuestro planeta.

Entendimiento de los datos

1. Se carga el dataset en el cuaderno de Google notebook



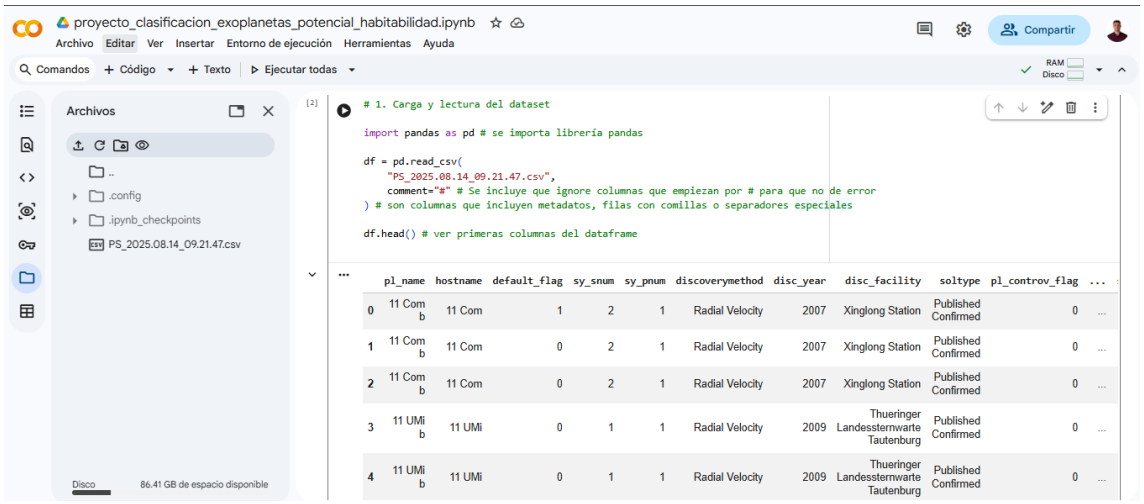


2. Se carga el dataset y se hace su lectura

```
# 1. Carga y lectura del dataset
import pandas as pd # se importa librería pandas
df = pd.read_csv(
    "PS_2025.08.14_09.21.47.csv",
    comment="#"
)
df.head() # ver primeras columnas del dataframe
```

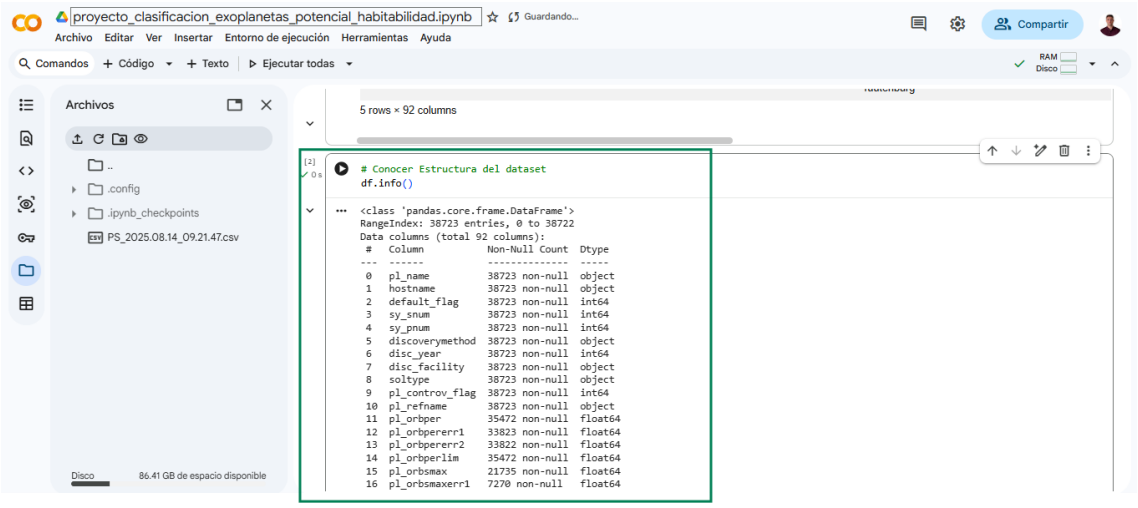
Es necesario indicar que no lea algunas columnas para evitar que aparezca un error al intentar procesar columnas que empiezan por un #. Estas columnas y filas se pueden obviar sin inconvenientes ya que contienen metadatos o separadores especiales. Específicamente estas filas y columnas que no se van a contemplar y que carecen de relevancia son:

- Fecha de generación del dataset
- Nombre de la consulta
- Versión del catálogo
- Fuente (NASA Exoplanet Archive)



3. Mediante el uso del siguiente código se conoce la estructura del dataset, se puede observar el número de columnas, los tipos de datos, que columnas contienen valores nulos…

```
# Conocer Estructura del dataset
df.info()
```



El dataset contiene principalmente variables numéricas que representan parámetros físicos y orbitales de los exoplanetas. Se observa la presencia de valores nulos en algunas columnas

4. Se seleccionan solo aquellas columnas interesantes para el objetivo que persigue el proyecto, poder clasificar que planetas tienen mayor probabilidad de albergar vida en base a sus similitudes terrestres. Las columnas elegidas están relacionadas con la capacidad de un planeta para mantener una atmósfera estable y condiciones compatibles con la vida tal como la conocemos.

- Radio del planeta:

  Indica el tamaño del planeta y permite diferenciar entre planetas rocosos, supertierras y gigantes gaseosos. Los planetas con radios similares al de la Tierra tienen mayor probabilidad de ser cuerpos rocosos con superficie sólida, condición necesaria para la existencia de agua líquida estable. Además, el radio influye en la capacidad del planeta para retener una atmósfera.

- Masa del planeta

  La masa planetaria es un factor clave para determinar la gravedad superficial del planeta.

- Temperatura equilibrio

  La temperatura de equilibrio es una estimación de la temperatura superficial del planeta basada en la energía recibida de su estrella y en su capacidad para reflejar dicha energía. Este parámetro es fundamental para evaluar si el planeta podría mantener agua líquida, uno de los criterios esenciales de habitabilidad.

- Flujo estelar

  El flujo estelar representa la cantidad de energía que el planeta recibe de su estrella en comparación con la que recibe la Tierra del Sol.

- Periodo orbital

  El periodo orbital describe el tiempo que el planeta tarda en completar una órbita alrededor de su estrella. Este valor permite inferir la distancia orbital y, por tanto, la posición del planeta respecto a la zona habitable de su sistema estelar. Planetas con periodos orbitales compatibles con dicha zona tienen mayor probabilidad de mantener condiciones favorables para la habitabilidad.

5. Como mejora para poder disponer de más datos y ser más precisos se pretenden crear estas dos columnas que hacen referencia a términos físicos interesantes para el estudio que persigue este análisis:

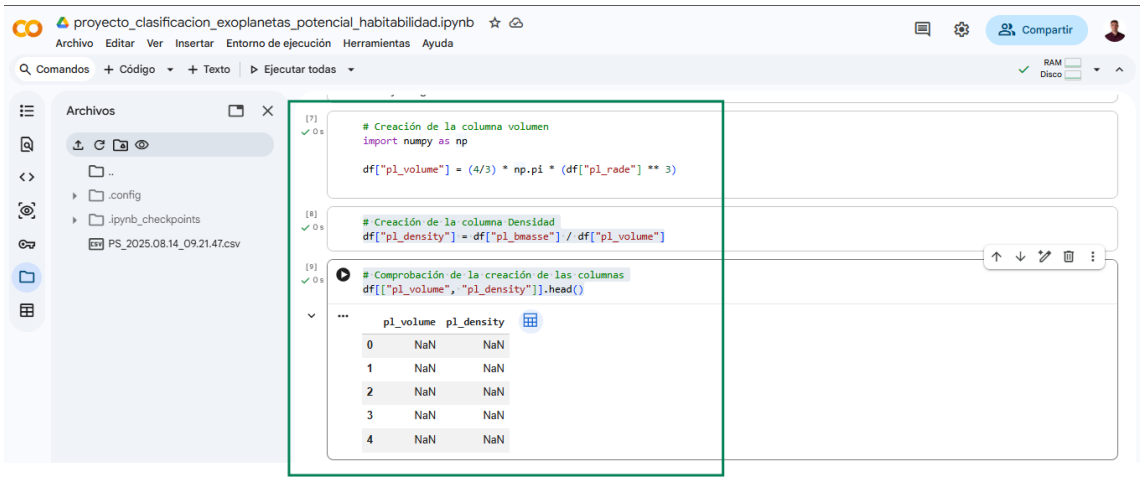
- Densidad

  La densidad permite inferir la composición interna del planeta. Densidades altas suelen indicar planetas rocosos, mientras que densidades bajas corresponden a planetas gaseosos.

- Volumen

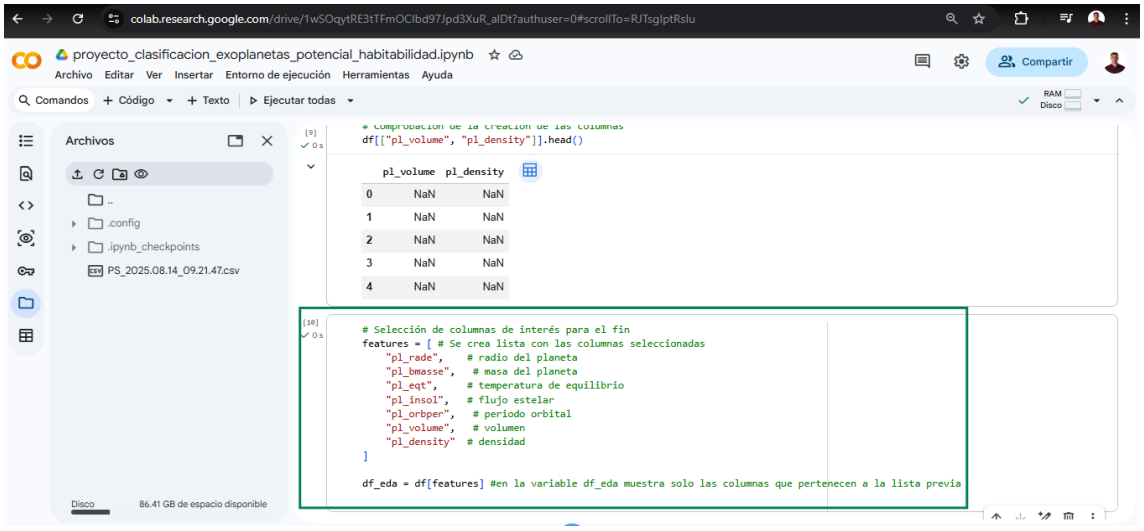
  El volumen de un planeta representa el espacio tridimensional que ocupa su cuerpo.

```
# Creación de la columna volumen
import numpy as np

df["pl_volume"] = (4/3) * np.pi * (df["pl_rade"] ** 3)

# Creación de la columna Densidad
df["pl_density"] = df["pl_bmasse"] / df["pl_volume"]

# Comprobación de la creación de las columnas
df[["pl_volume", "pl_density"]].head()
```



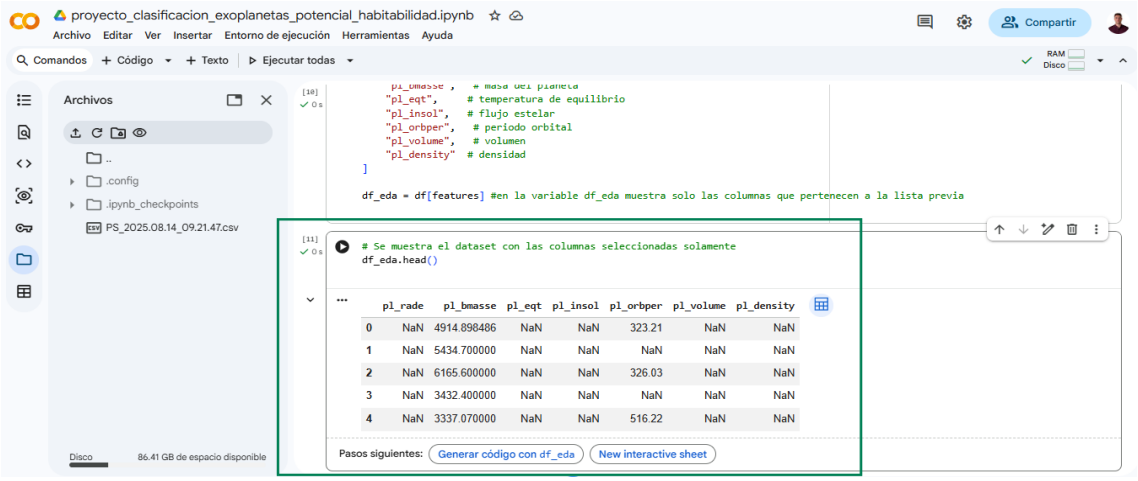
6. Una vez se han creado estas 2 nuevas columnas se seleccionan todas las de relevancia e interés para disponer de la información necesaria para elaborar el modelo posteriormente.

```
# Selección de columnas de interés para el fin

features = [ # Se crea lista con las columnas seleccionadas

    "pl_rade",     # radio del planeta

    "pl_bmasse",    # masa del planeta

    "pl_eqt",      # temperatura de equilibrio

    "pl_insol",    # flujo estelar

    "pl_orbper",    # periodo orbital

    "pl_volume",    # volumen

    "pl_density"  # densidad

]

df_eda = df[features] # en la variable df_eda muestra solo las columnas que pertenecen a la lista previa
```



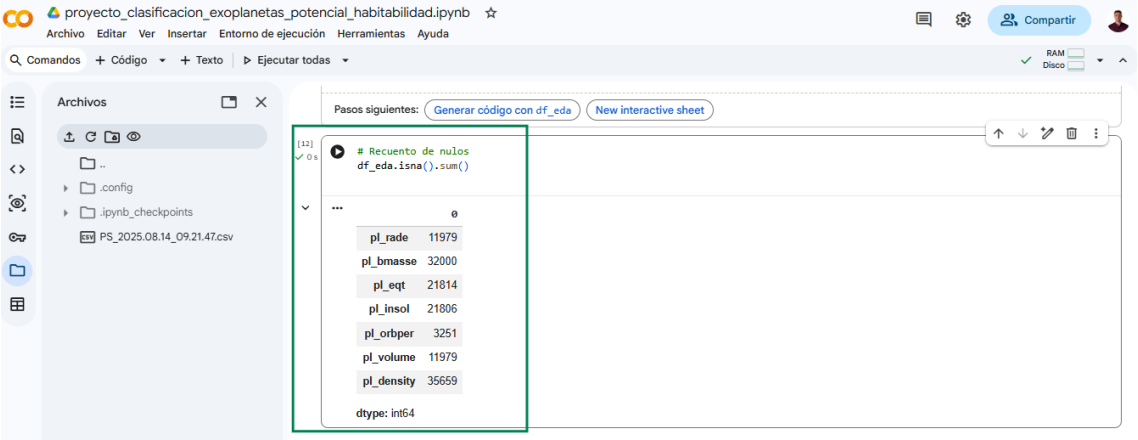
7. Se muestra por pantalla el dataset con solo las primeras columnas de interés mencionadas antes.

```
# Se muestra el dataset con las columnas seleccionadas solamente
df_eda.head()
```



8. Se realiza un conteo de valores nulos en el df_eda

df_eda.isna().sum()

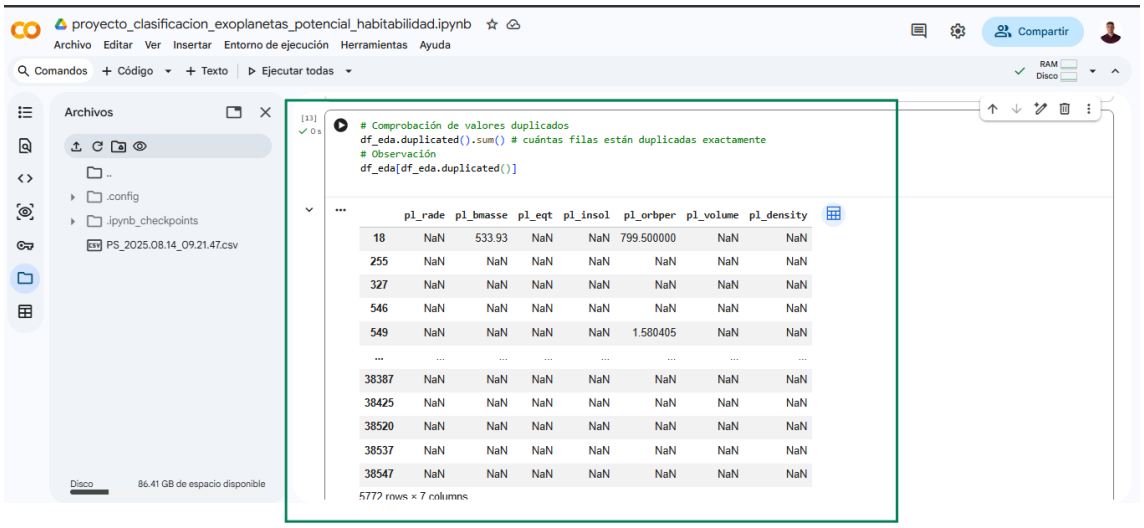


Variables como el radio planetario (pl_rade) y la masa mínima (pl_bmasse) presentan un elevado número de valores ausentes debido a la dependencia de distintos métodos de detección. Como consecuencia, las variables derivadas, volumen (pl_volume) y densidad (pl_density), presentan valores nulos cuando no se dispone de las magnitudes necesarias para su cálculo.

Estos valores no representan errores, sino la ausencia de mediciones directas necesarias para su estimación, una situación común en estudios de exoplanetas.

9. Se realiza una comprobación de valores duplicados

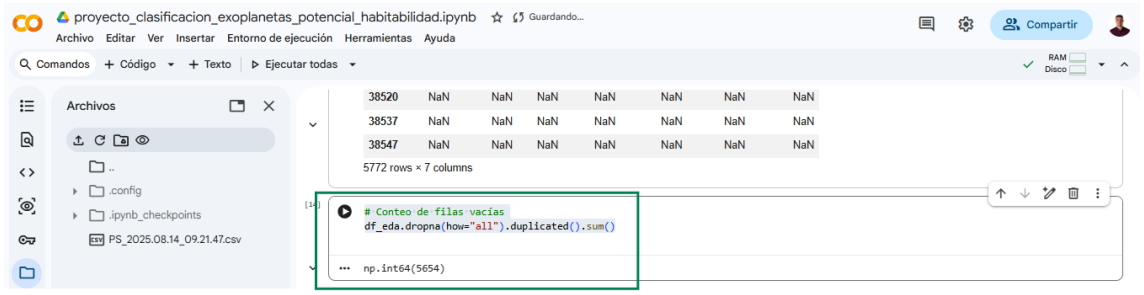
```
# Comprobación de valores duplicados
df_eda.duplicated().sum() # cuántas filas están duplicadas
exactamente
# Observación
df_eda[df_eda.duplicated()]
```



Aparecen valores duplicados pero no es problemático ya que muchas filas contienen “NaN” que lo que indica es que no se dispone de ese valor, es decir, no se dispone de una medición para ese campo.

Se realiza una comprobación adicional contando las filas vacías

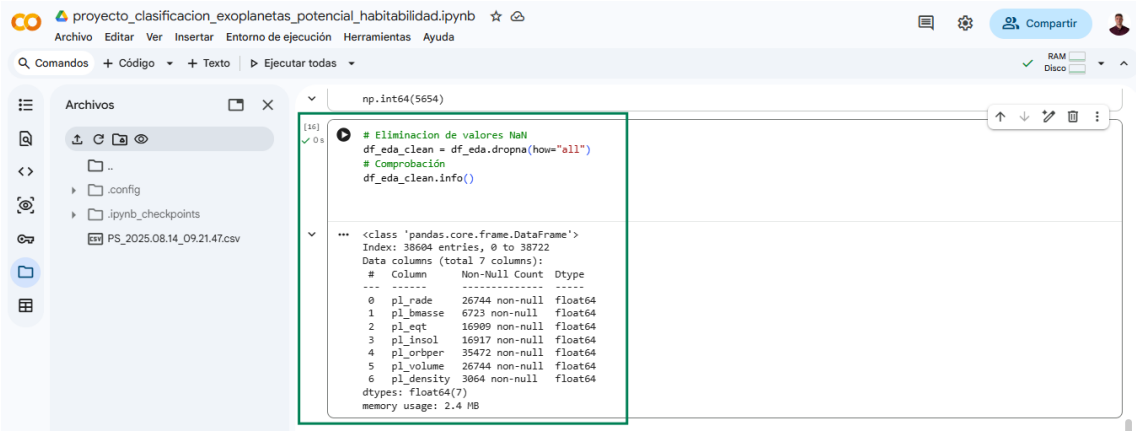
```
# Conteo de filas vacías

df_eda.dropna(how="all").duplicated().sum()
```



Se identifican 5654 registros que no contienen información en ninguna de las variables seleccionadas.

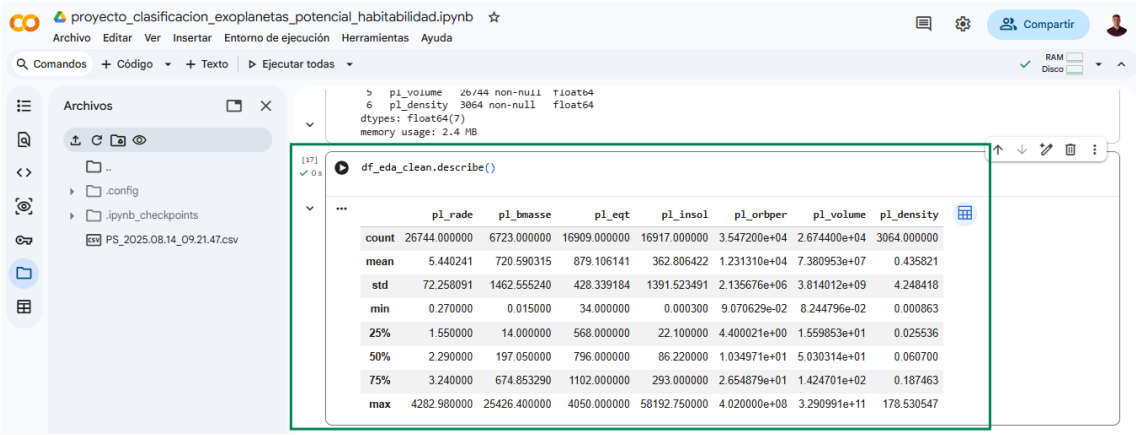
10. Se genera un nuevo dataframe que almacene todas las columnas de interés pero dejando de lado aquellos valores que son NaN.

```
# Eliminacion de valores NaN
df_eda_clean = df_eda.dropna(how="all")
# Comprobación
df_eda_clean.info()
```



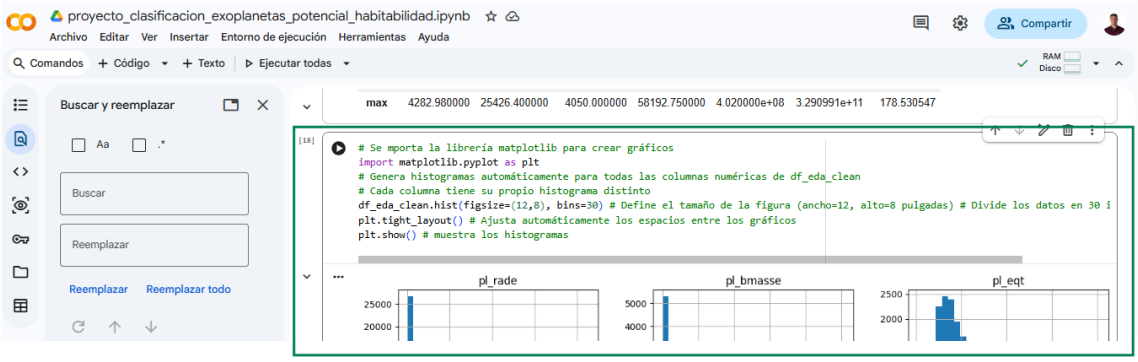
11. Se inicia el apartado de estadísticas descritivas

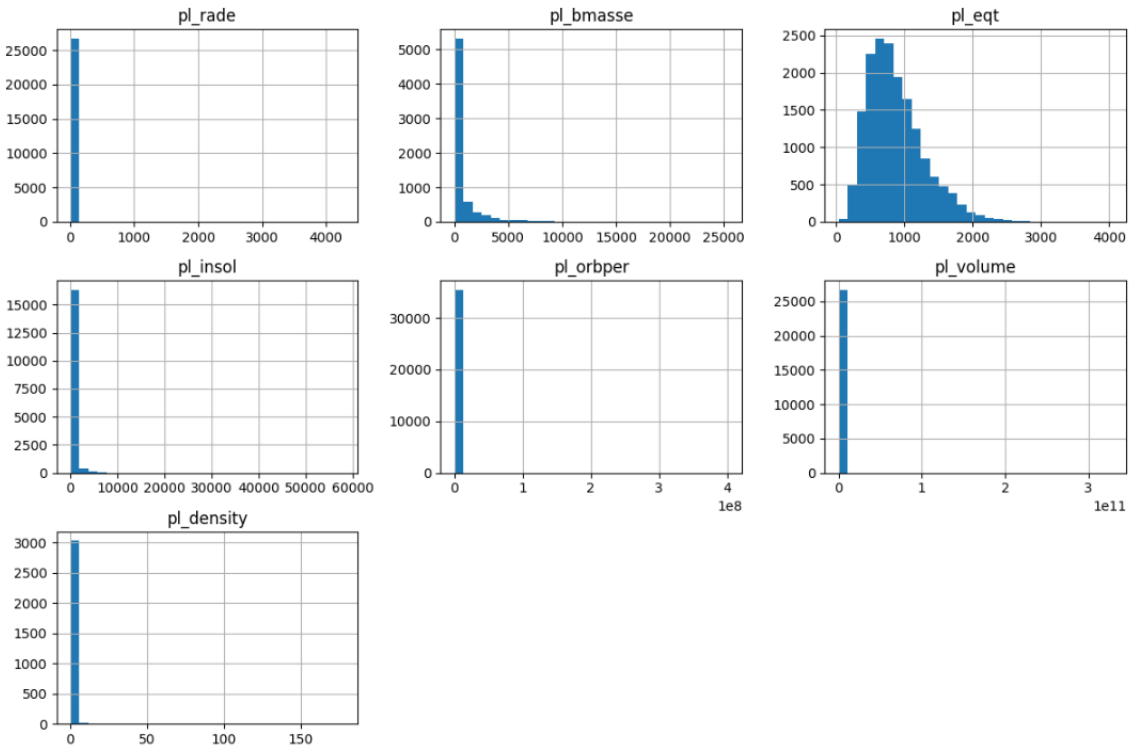
```
df_eda_clean.describe()
```



	pl_rade	pl_bmasse	pl_eqt	pl_insol	pl_orbper	pl_volume	pl_density
count	26744.000000	6723.000000	16909.000000	16917.000000	3.547200e+04	2.674400e+04	3064.000000
mean	5.440241	720.590315	879.106141	362.806422	1.231310e+04	7.380953e+07	0.435821
std	72.258091	1462.555240	428.339184	1391.523491	2.135676e+06	3.814012e+09	4.248418
min	0.270000	0.015000	34.000000	0.000300	9.070629e-02	8.244796e-02	0.000863
25%	1.550000	14.000000	568.000000	22.100000	4.400021e+00	1.559853e+01	0.025536
50%	2.290000	197.050000	796.000000	86.220000	1.034971e+01	5.030314e+01	0.060700
75%	3.240000	674.853290	1102.000000	293.000000	2.654879e+01	1.424701e+02	0.187463
max	4282.980000	25426.400000	4050.000000	58192.750000	4.020000e+08	3.290991e+11	178.530547

Mediante el análisis de estadísticas descriptivas se observa una elevada dispersión en los valores de las variables seleccionadas. Esta variabilidad refleja la diversidad de exoplanetas detectados, abarcando desde planetas pequeños y fríos hasta cuerpos masivos con altos niveles de irradiación estelar.





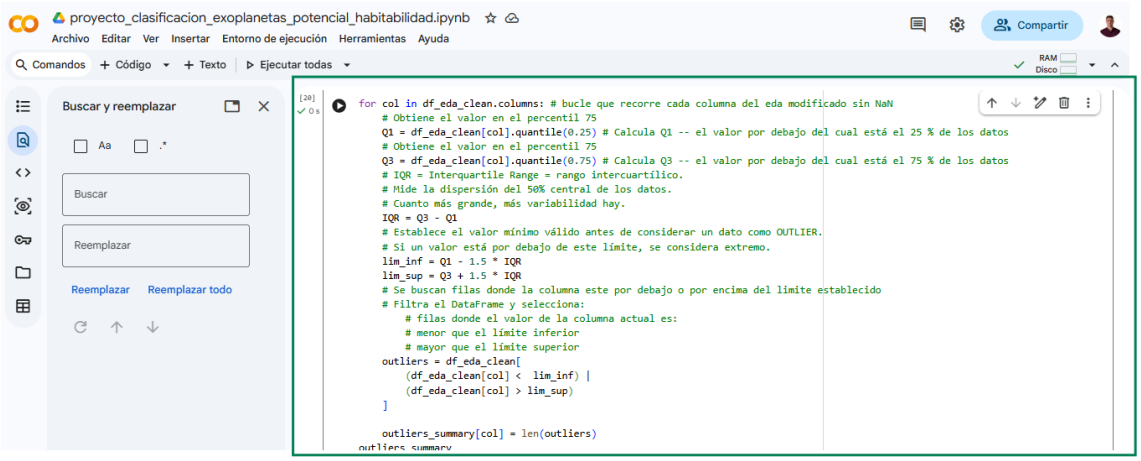
Las distribuciones de las variables presentan asimetrías pronunciadas, especialmente en la masa, el volumen y el periodo orbital, lo cual es característico de los datos astronómicos. Estos resultados sugieren la necesidad de aplicar técnicas de escalado antes del modelado.

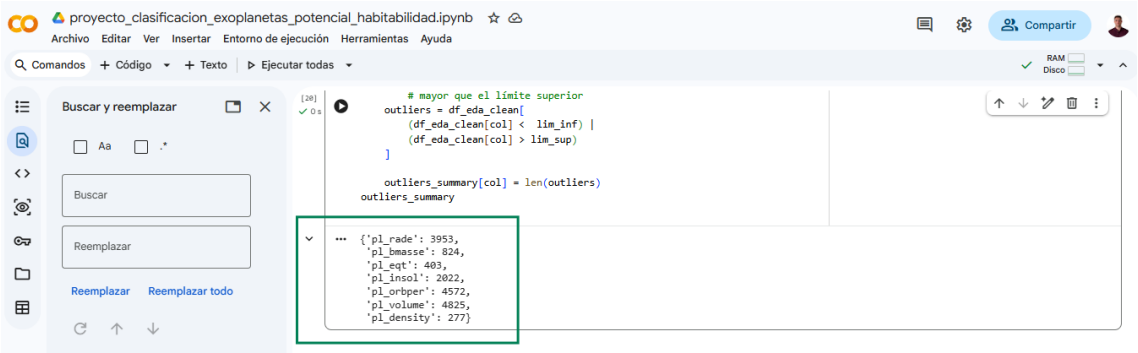
12. Identificación de outliders

Los outliers pueden representar errores de medición, unidades mal registradas o condiciones extremas. Identificarlos es útil para entender la variabilidad de los datos y decidir si deben ser tratados o conservados.

```
# Detección de Outliders

outliers_summary = {} # Se crea diccionario vacío donde se almacena

            # clave -- nombre de la columna

            # valor -- número de outliers en esa columna

for col in df_eda_clean.columns: # bucle que recorre cada columna del eda modificado sin NaN

  # Obtiene el valor en el percentil 75

  Q1 = df_eda_clean[col].quantile(0.25) # Calcula Q1 -- el valor por debajo del cual está el 25 % de los datos

    # Obtiene el valor en el percentil 75

  Q3 = df_eda_clean[col].quantile(0.75) # Calcula Q3 -- el valor por debajo del cual está el 75 % de los datos

    # IQR = Interquartile Range = rango intercuartílico.

    # Mide la dispersión del 50% central de los datos.
```

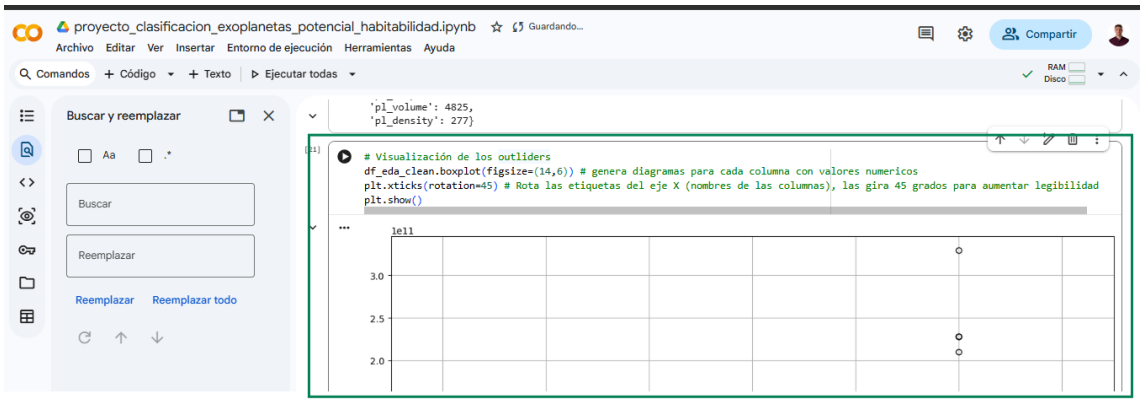
```
    # Cuanto más grande, más variabilidad hay.
  IQR = Q3 - Q1
  # Establece el valor mínimo válido antes de considerar un dato como OUTLIER.
  # Si un valor está por debajo de este límite, se considera extremo.
  lim_inf = Q1 - 1.5 * IQR
  lim_sup = Q3 + 1.5 * IQR
  # Se buscan filas donde la columna este por debajo o por encima del limite establecido
    # Filtra el DataFrame y selecciona:
        # filas donde el valor de la columna actual es:
        # menor que el límite inferior
        # mayor que el límite superior
  outliers = df_eda_clean[
    (df_eda_clean[col] <  lim_inf) |
    (df_eda_clean[col] > lim_sup)
  ]
  outliers_summary[col] = len(outliers)
outliers_summary
```

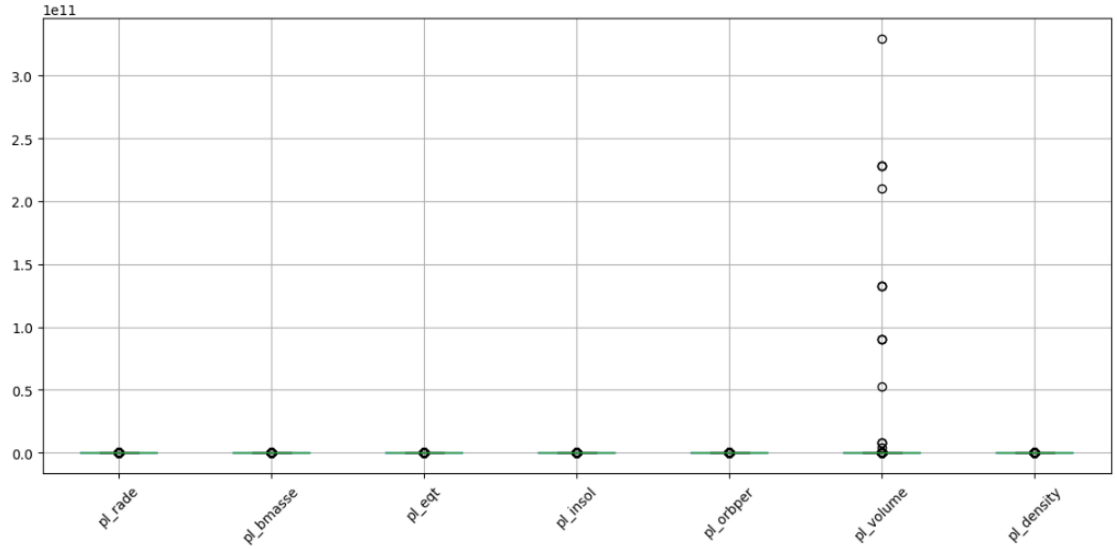




La detección de valores extremos mediante el método del rango intercuartílico revela la presencia de un número significativo de outliers en variables como el radio, el volumen y el periodo orbital. Estos valores no representan errores en el conjunto de datos, sino planetas con características físicas extremas, como gigantes gaseosos o cuerpos con órbitas muy cercanas o lejanas a sus estrellas. Por este motivo, los outliers se mantienen en el análisis para preservar la realidad y diversidad física de los cuerpos celestes que permiten establecer una clasificación real en base a las similitudes terrestres.

```
# Visualización de los outliders
df_eda_clean.boxplot(figsize=(14,6)) # genera diagramas para cada columna con valores numericos
plt.xticks(rotation=45) # Rota las etiquetas del eje X (nombres de las columnas), las gira 45 grados para aumentar legibilidad
plt.show() # lo muestra
```



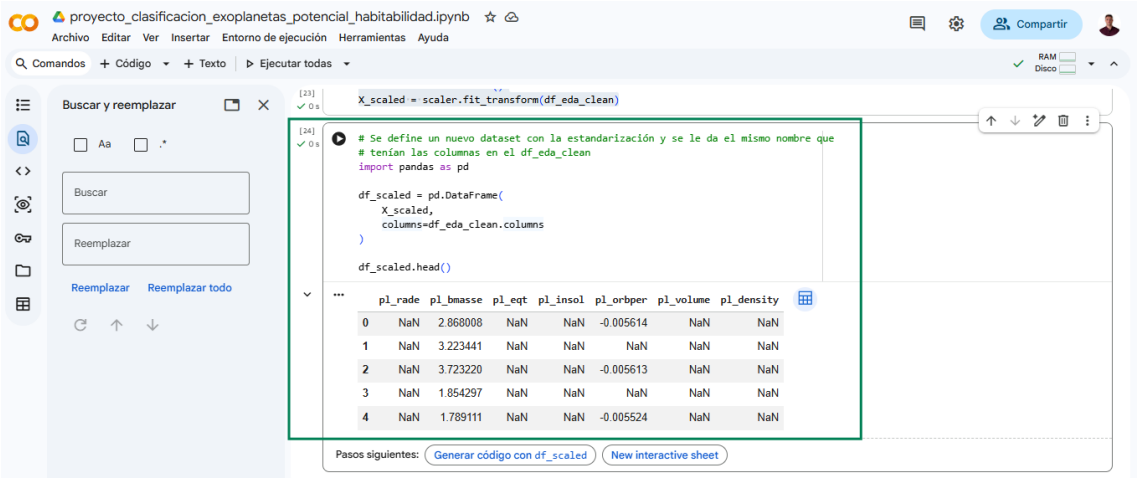


Se utilizaron diagramas de caja para identificar visualmente la presencia de valores extremos y analizar la dispersión de las variables físicas seleccionadas.

Preparación de los datos

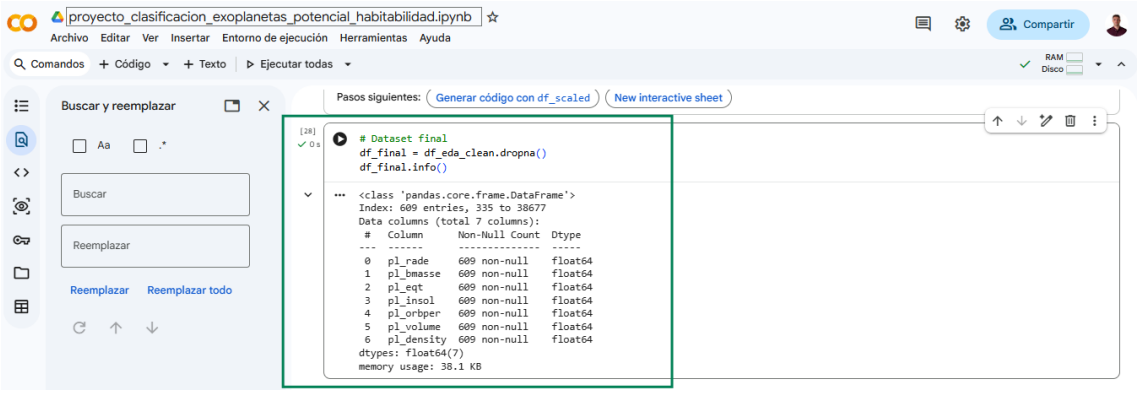
1. Dado que las variables presentan distintos órdenes de magnitud, se aplicó un proceso de estandarización para evitar que unas variables dominen sobre otras durante el entrenamiento.

```
# Estandarización de los valores de las columnas

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(df_eda_clean)

# Se define un nuevo dataset con la estandarización y se le da el
mismo nombre que tenían las columnas en el df_eda_clean

import pandas as pd

df_scaled = pd.DataFrame(

    X_scaled,

    columns=df_eda_clean.columns

)

df_scaled.head()
```



2. Se crea un último dataframe a partir del anterior df_eda_clean eliminando todos aquellos planetas que no tienen todos los datos completos, es decir, en algunas de sus columnas tienen un valor igual a NaN.

```
# Dataset final

df_final = df_eda_clean.dropna()

df_final.info()
```

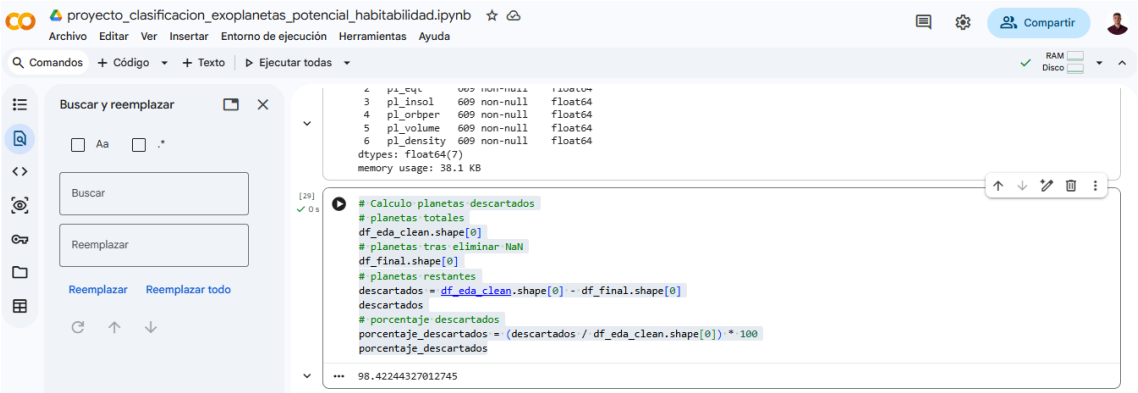


3. Se calcula el numero de planetas que han sido descartados al no tener valores en todas las columnas con el objetivo de determinar si se tienen suficientes datos para realizar el modelo.

```
# Calculo planetas descartados

# planetas totales

df_eda_clean.shape[0]

# planetas tras eliminar NaN

df_final.shape[0]

# planetas restantes

descartados = df_eda_clean.shape[0] - df_final.shape[0]

descartados

# porcentaje descartados
```

porcentaje_descartados = (descartados / df_eda_clean.shape[0]) * 100

porcentaje_descartados



Se han descartado un 98% de los planetas al no tener todos los valores suficientes para realizar un correcto análisis. Esta decisión garantiza que el modelo se entrene exclusivamente con planetas cuyas propiedades físicas han sido caracterizadas de forma completa, evitando la introducción de sesgos derivados de la imputación de datos ausentes. Se ha priorizado recabar datos de planetas bien observados, cercanos y con múltiples métodos de detección.

4. Se calcula el número total de planetas que quedan en el dataframe que se va a utilizar para el entrenamiento del modelo.

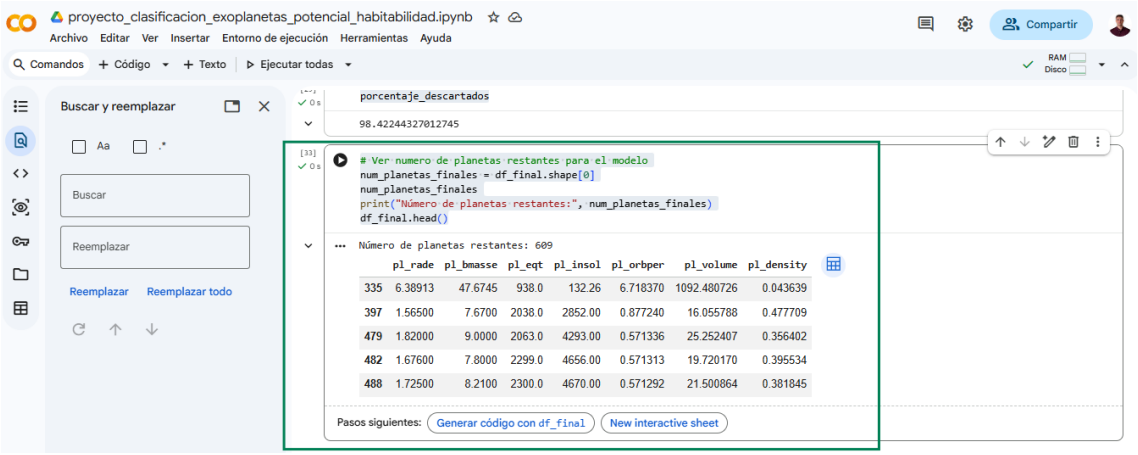
```
# Ver número de planetas restantes para el modelo
```

num_planetas_finales = df_final.shape[0]

num_planetas_finales

print("Número de planetas restantes:", num_planetas_finales)

df_final.head()

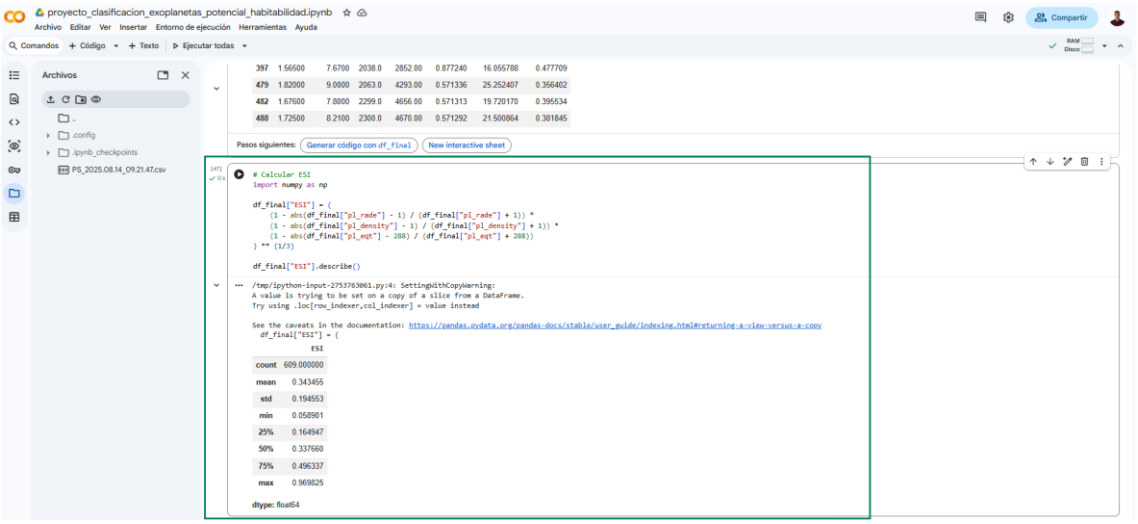


Modelado

1. Se calcula el Earth Similarity Index (ESI), ya que constituye una métrica ampliamente utilizada para cuantificar la similitud de un planeta con respecto a las condiciones terrestres. A partir de este índice se define la variable objetivo del modelo, permitiendo clasificar los exoplanetas según su grado de similitud con la Tierra.

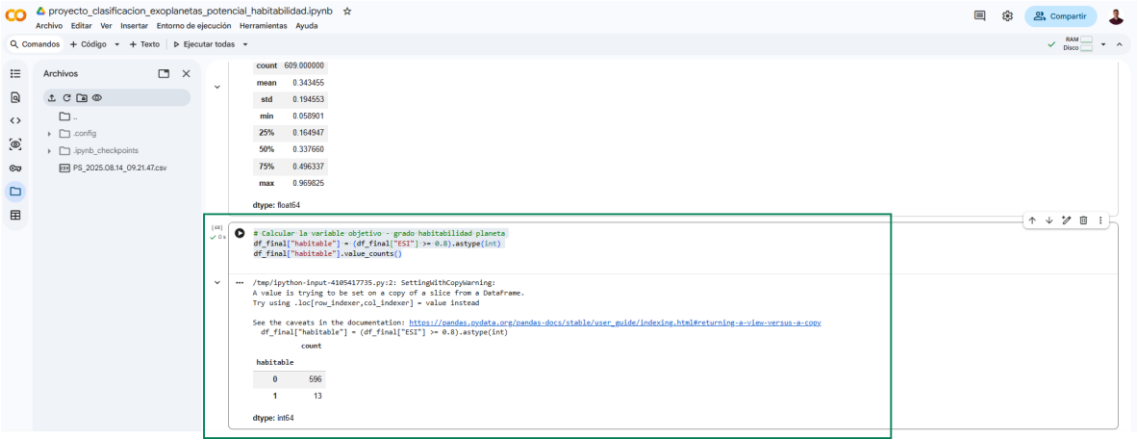
Para poder definir más tarde la variable objetivo del modelo, se crea la variable Earth Similarity Index (ESI) a partir de parámetros físicos clave. Posteriormente, se estableció un umbral de clasificación, considerando como potencialmente habitables aquellos planetas con un valor de ESI igual o superior a 0.8.

```
import numpy as np

df_final["ESI"] = (

    (1 - abs(df_final["pl_rade"] - 1) / (df_final["pl_rade"] + 1))
*

    (1 - abs(df_final["pl_density"] - 1) / (df_final["pl_density"]
+ 1)) *

    (1 - abs(df_final["pl_eqt"] - 288) / (df_final["pl_eqt"] +
288))

) ** (1/3)

df_final["ESI"].describe()
```



2. Earth Similarity Index (ESI) se utiliza como una métrica de referencia para cuantificar la similitud de un planeta con respecto a la Tierra. A partir de este índice se define la variable objetivo del modelo, denominada *habitabilidad*, que permite formular el problema como una tarea de clasificación supervisada.

```
# Calcular la variable objetivo - grado habitabilidad planeta

df_final["habitable"] = (df_final["ESI"] >= 0.8).astype(int)

df_final["habitable"].value_counts()
```



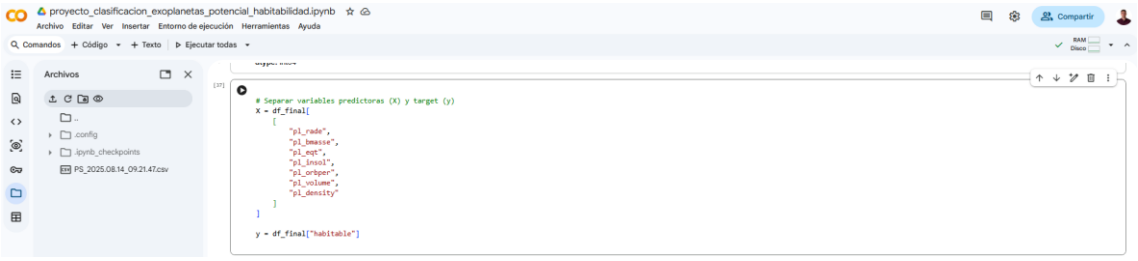
3. Se separan las variables predictoras (x) y target (y).

En un modelo de Machine Learning se necesita saber: qué información usar para aprender y qué es lo que debe predecir.

X, son los datos de entrada del modelo que va a usar para aprender, en el caso: radio del planeta, masa, densidad, temperatura equilibrio, flujo estelar, periodo orbital, densidad y volumen.

La Y, Es el resultado que quieres que el modelo aprenda a predecir, es decir, el grado de habitabilidad de cada planeta, nuestra variable objetivo *habitabilidad*.

```
X = df_final[
    [
        "pl_rade",
        "pl_bmasse",
        "pl_eqt",
        "pl_insol",
        "pl_orbper",
        "pl_volume",
        "pl_density"
    ]
]
y = df_final["habitable"]
```

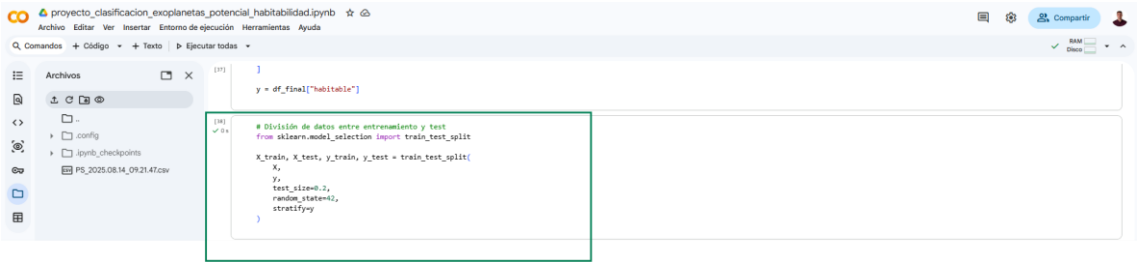


5. Se hace una separación de los datos para poder realizar el entrenamiento del modelo:

- X_train: variables predictoras para entrenar
- X_test: variables predictoras para evaluar
- y_train: etiquetas reales para entrenar
- y_test: etiquetas reales para evaluar

Se emplea un 80% de los datos para realizar el entrenamiento y el 20% de los datos se deja para realizar la parte de test del modelo. Al tener 608 planetas, se usan 487 para entrenar y 122 para evaluar.

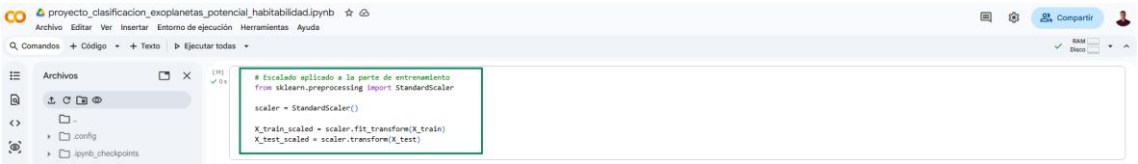
```
# División de los datos entre entrenamiento y test

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42,

    stratify=y

)
```



6. Escalado aplicado a la parte de entrenamiento

Dado que las variables predictoras presentan distintos órdenes de magnitud, se aplicó un proceso de estandarización mediante StandardScaler. El escalado se ajustó exclusivamente sobre el conjunto de entrenamiento y posteriormente se aplicó al conjunto de prueba,

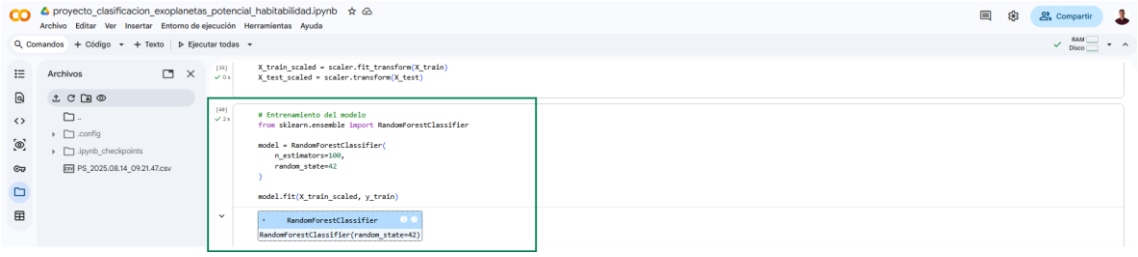
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```



7. Entrenamiento del modelo

Se entrena el modelo de Random Forest para que aprenda la relación entre los parámetros físicos (X) y la variable objetivo (*habitabilidad*).

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(

    n_estimators=100,

    random_state=42

)

model.fit(X_train_scaled, y_train)
```



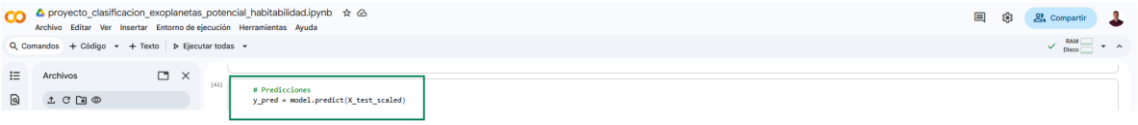
8. Predicciones

El modelo entrenado se utiliza para predecir la clase de habitabilidad sobre planetas no vistos durante el entrenamiento. Estas predicciones se comparan con y_test para evaluar el rendimiento.

y_pred contiene:

- 0 → no habitable
- 1 → potencialmente habitable

```
y_pred = model.predict(X_test_scaled)
```

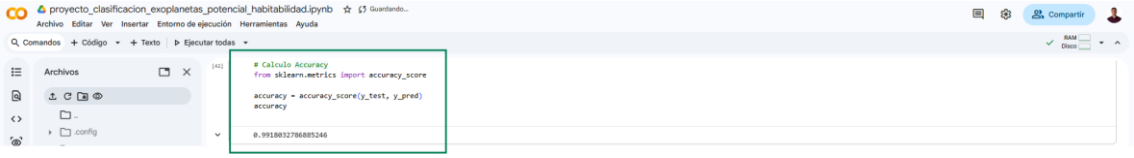


Evaluación

Se realiza el calculo de las siguientes magnitudes y la matriz de confusión para evaluar lo adecuado del modelo creado.

1. Calculo Accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

accuracy
```



2. Calculo Recall, F1-Score

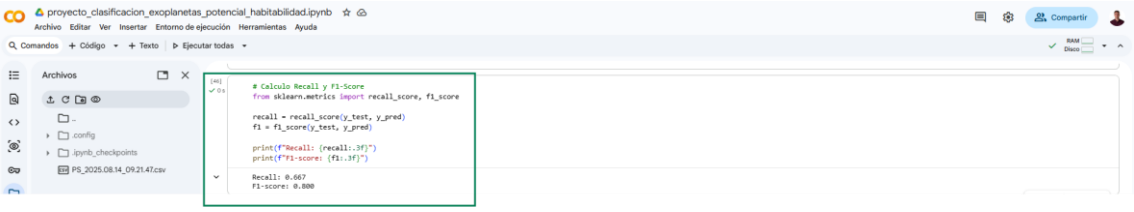
from sklearn.metrics import recall_score, f1_score

recall = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

print(f"Recall: {recall:.3f}")

print(f"F1-score: {f1:.3f}")



Dado el desbalance entre clases, se analizaron métricas adicionales al accuracy, en particular el recall y el F1-score. El recall resulta especialmente relevante en este contexto, ya que mide la capacidad del modelo para identificar correctamente exoplanetas potencialmente habitables, minimizando los falsos negativos. El F1-score proporciona una medida equilibrada del rendimiento global del modelo, combinando precisión y recall.

Se obtuvo un Recall = 0.667. Esto significa que ll modelo detecta correctamente aproximadamente el 66,7 % de los planetas realmente potencialmente habitables.

En términos prácticos:

- De cada 3 planetas habitables, el modelo identifica 2
- 1 se le escapa (falso negativo)

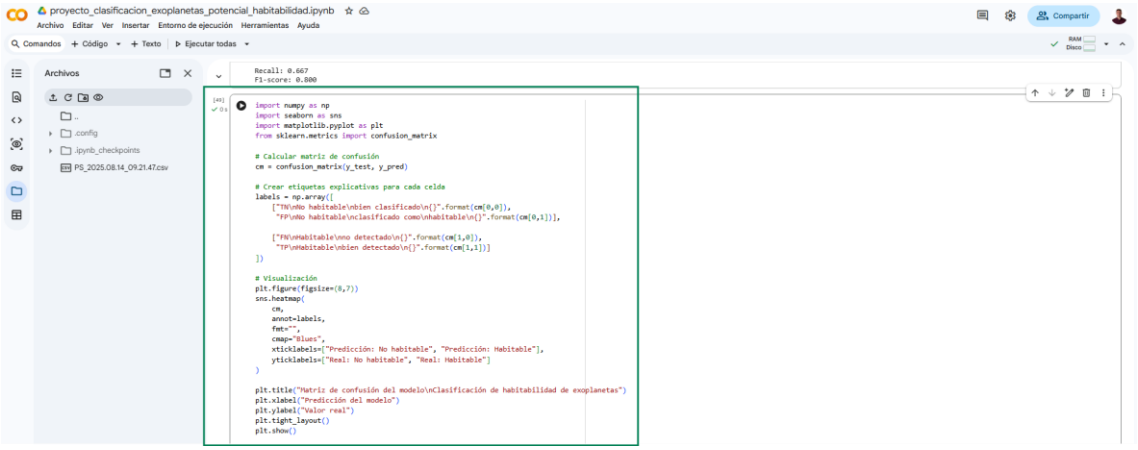
Se obtruvo un F1-score = 0.800. Esto implica que existe un buen equilibrio entre la capacidad de detección (recall) y la precisión del modelo.

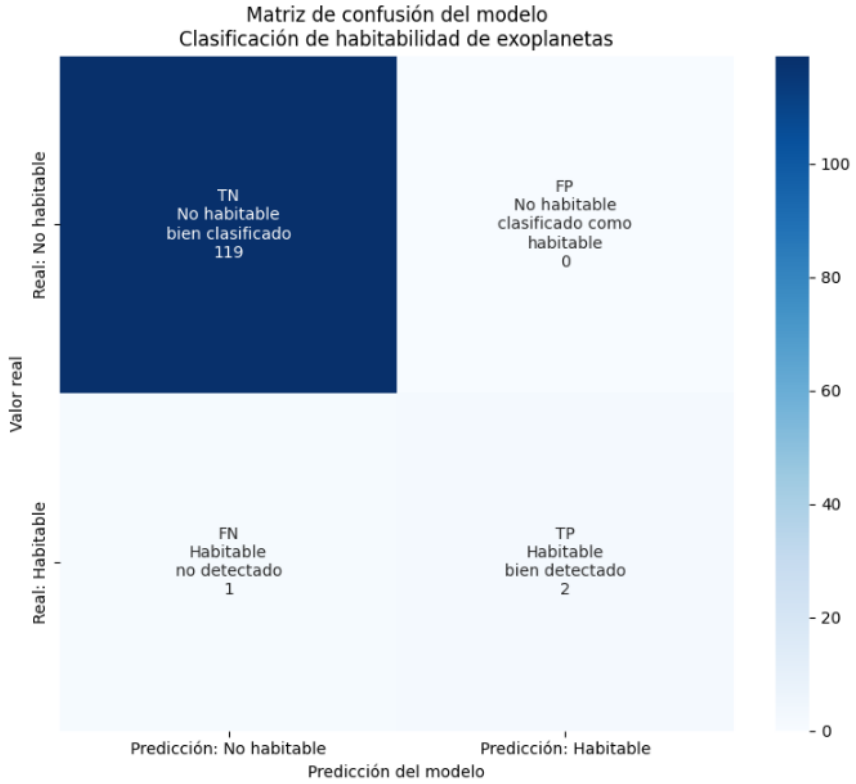
3. Matriz de confusión

Permite entender como se equivoca el modelo

- TN → No habitables bien clasificados
- FP → Planetas clasificados como habitables, pero no lo son
- FN → Planetas habitables que el modelo no detectó
- TP → Planetas habitables correctamente detectados

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Calcular matriz de confusión
cm = confusion_matrix(y_test, y_pred)

# Crear etiquetas explicativas para cada celda
labels = np.array([
    ["TN\nNo habitable\nbien clasificado\n{}".format(cm[0,0]),
     "FP\nNo habitable\nclasificado
como\nhabitable\n{}".format(cm[0,1])],

    ["FN\nHabitable\nno detectado\n{}".format(cm[1,0]),
     "TP\nHabitable\nbien detectado\n{}".format(cm[1,1])]
])

# Visualización
plt.figure(figsize=(8,7))
sns.heatmap(
    cm,
    annot=labels,
    fmt="",
    cmap="Blues",
    xticklabels=["Predicción: No habitable", "Predicción:
Habitable"],
    yticklabels=["Real: No habitable", "Real: Habitable"]
)

plt.title("Matriz de confusión del modelo\nClasificación de
habitabilidad de exoplanetas")
plt.xlabel("Predicción del modelo")
plt.ylabel("Valor real")
plt.tight_layout()
plt.show()
```





La matriz de confusión permite analizar en detalle el comportamiento del modelo. Se observa que el modelo clasifica correctamente la mayoría de los planetas no habitables y es capaz de identificar planetas potencialmente habitables.

Un valor alto de TN indica que el modelo distingue correctamente planetas claramente no similares a la Tierra.

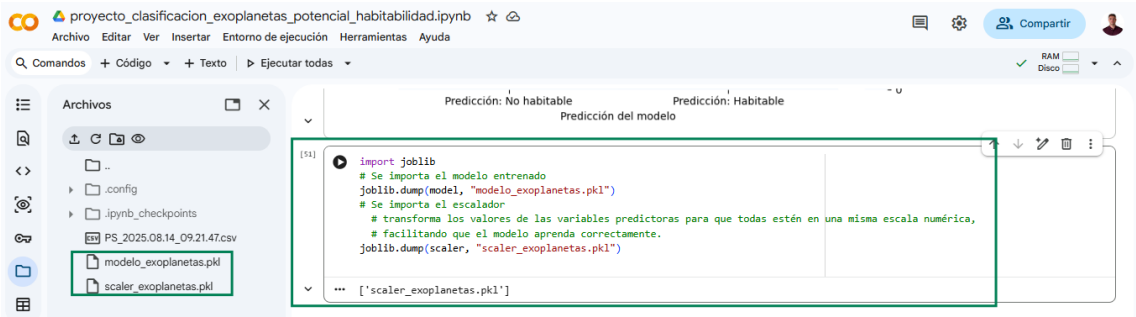
Deployment

1. Se guarda el modelo entrenado y el escalador para poder usarlos fuera del notebook

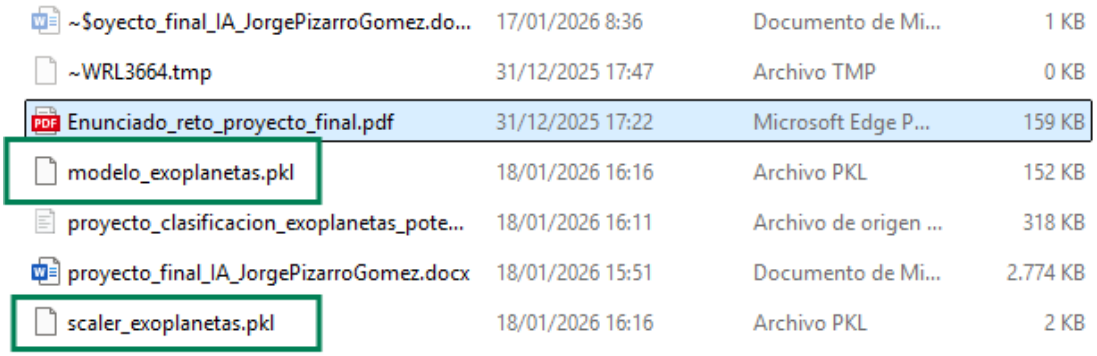
import joblib

joblib.dump(model, "modelo_exoplanetas.pkl")

joblib.dump(scaler, "scaler_exoplanetas.pkl")

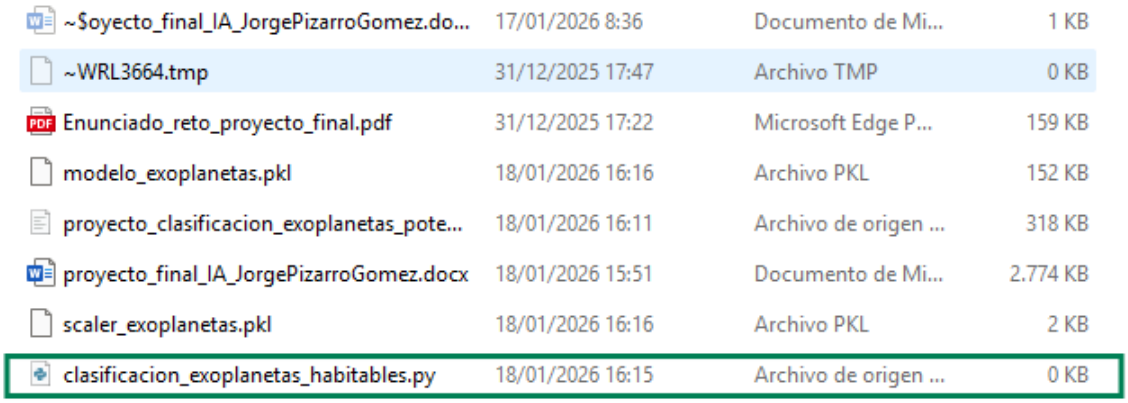


2. Del notebook se descargan estos dos nuevos archivos .pkl



~$oyecto_final_IA_JorgePizarroGomez.do...	17/01/2026 8:36	Documento de Mi...	1 KB
~WRL3664.tmp	31/12/2025 17:47	Archivo TMP	0 KB
Enunciado_reto_proyecto_final.pdf	31/12/2025 17:22	Microsoft Edge P...	159 KB
modelo_exoplanetas.pkl	18/01/2026 16:16	Archivo PKL	152 KB
proyecto_clasificacion_exoplanetas_pote...	18/01/2026 16:11	Archivo de origen ...	318 KB
proyecto_final_IA_JorgePizarroGomez.docx	18/01/2026 15:51	Documento de Mi...	2.774 KB
scaler_exoplanetas.pkl	18/01/2026 16:16	Archivo PKL	2 KB

3. Se crea un archivo flask clasificacion_exoplanetas_habitables.py almacenar el modelo



~$oyecto_final_IA_JorgePizarroGomez.do...	17/01/2026 8:36	Documento de Mi...	1 KB
~WRL3664.tmp	31/12/2025 17:47	Archivo TMP	0 KB
Enunciado_reto_proyecto_final.pdf	31/12/2025 17:22	Microsoft Edge P...	159 KB
modelo_exoplanetas.pkl	18/01/2026 16:16	Archivo PKL	152 KB
proyecto_clasificacion_exoplanetas_pote...	18/01/2026 16:11	Archivo de origen ...	318 KB
proyecto_final_IA_JorgePizarroGomez.docx	18/01/2026 15:51	Documento de Mi...	2.774 KB
scaler_exoplanetas.pkl	18/01/2026 16:16	Archivo PKL	2 KB
clasificacion_exoplanetas_habitables.py	18/01/2026 16:15	Archivo de origen ...	0 KB

4. Se rellena el archivo .py con el código necesario desde VSC

```
# Flask --> crea la aplicación web
# request → permite recibir datos que envía el usuario (JSON)
# jsonify → convierte la respuesta de Python en JSON, que es el formato
estándar en APIs
from flask import Flask, request, jsonify
# joblib sirve para cargar objetos guardados previamente. En el caso el
modelo y el escalador
import joblib
import numpy as np

# Crear aplicación Flask
app = Flask(__name__)

# Cargar modelo y escalador
model = joblib.load("modelo_exoplanetas.pkl")
scaler = joblib.load("scaler_exoplanetas.pkl")

# ENDPOINT Principal
@app.route("/") # define ruta accesible desde el navegador
def home(): # al acceder devuelve ese texto
    return "API de clasificación de habitabilidad de exoplanetas"

# Define un ENDPOINT llamado /predict que solo acepta peticiones post
    # POST --> para enviar datos al servidor
@app.route("/predict", methods=["POST"])
def predict(): # funcion que se ejecuta cuando alguien llama a /predict
    data = request.get_json() # obtiene los datos enviados por el usuario
en formato json y se guardan en el diccionario data

    features = np.array([[
        data["pl_rade"],
        data["pl_bmasse"],
        data["pl_eqt"],
        data["pl_insol"],
        data["pl_orbper"],
        data["pl_volume"],
        data["pl_density"]
    ]])

   features_scaled = scaler.transform(features) # apliica el mismo
escalado que se uso para el entrenamiento
    prediction = model.predict(features_scaled)[0] # el modelo hace la
prediccion
    return jsonify({ # respuesta que se da al usuauario
        "habitabilidad": int(prediction),
       "resultado": "Potencialmente habitable" if prediction == 1 else
"No habitable"
```

```
    })

if __name__ == "__main__":
    app.run(debug=True)
```

El despliegue del modelo se realizó mediante un script Python independiente (clasificacion_exoplanetas_habitables.py) que implementa una aplicación Flask. Este script carga el modelo entrenado y el escalador previamente serializados y expone un endpoint REST (Una dirección (URL) a la que se puede enviar una petición para ejecutar una acción y obtener una respuesta) que permite clasificar nuevos exoplanetas a partir de sus parámetros físicos.

Bibliografía

https://phl.upr.edu/projects/earth-similarity-index-esi

https://arxiv.org/abs/1801.07101

https://www.mdpi.com/2075-4434/12/6/86

https://astrobiology.com/2023/05/assessing-exoplanet-habitability-through-data-driven-approaches-a-comprehensive-literature-review.html